

Linear Regression & Linear Classification

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- Reframe fitting a linear model as **training**: iteratively adapting parameters using a loss, not solving for them in one shot
- Explain why we use an iterative approach here even though linear regression has a closed-form solution
- Compare full-batch gradient descent, stochastic gradient descent, and mini-batch gradient descent, and describe the trade-offs between them
- Connect linear classification's loss to the same “loss + gradient” machinery as linear regression

The problem, again

What we already have

- A dataset of examples: pairs $(\mathbf{x}, y)$
- A model with parameters we can tune:

$$\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$$

- A way to score any choice of $(\mathbf{w}, b)$: mean squared error
- All three pieces came from last lecture

So what's actually missing?

- Last time, we picked (w, b) by hand, or let a random walk stumble into something decent
- A real dataset can have thousands of points and dozens of features — eyeballing it, or getting lucky, doesn't scale
 - As we add more dimensions we need more weights — w gets bigger, and a random step is decreasingly likely to land in a direction that actually helps
- We need a **systematic, automatic** way to adapt (w, b) toward a lower loss

That process — adapting parameters using data and a loss — is what **training** a model means.

Why not just solve it directly?

- Linear regression actually *has* a closed-form solution — the normal equation:

$$\mathbf{w}^{\star} = (\mathbf{X}^{\top} \mathbf{X})^{-1} \mathbf{X}^{\top} \mathbf{y}$$

- So why not just... do that?

Note

This folds $\mathbf{b}$ into $\mathbf{w}$ by adding a column of 1s to $\mathbf{X}$ — a standard convention, but new notation if you haven't seen it before.

Four ways to adapt (w , b)

We'll watch the same dataset get fit four different ways:

1. **Random search** — nudge and hope (already met this one)
2. **Gradient descent** — use *all* the data to take one confident step
3. **Stochastic gradient descent** — use just *one* point per step
4. **Mini-batch gradient descent** — a tunable middle ground

Random search, again

Random search, live

- We're scoring every guess against all **40** data points here — each “better or worse” call means running the whole dataset through the model
- Real datasets can run to terabytes; 40 rows is tiny by comparison — and that per-guess cost scales linearly with however much data we have



Still no sense of direction

- Same conclusion as before: blue = improved, red = made it worse — no way to tell in advance which a step will be
- With noisier data, the random walk struggles even more — more room to wander before stumbling onto a decent fit
- This is our baseline for the rest of the lecture — everything from here on should beat it

Remember, each time we say “better or worse” we need to do calculations!